



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**
Acreditación Institucional en Alta Calidad



CNN en OpenACC

Convolución 2D + ReLU + MaxPooling 2D

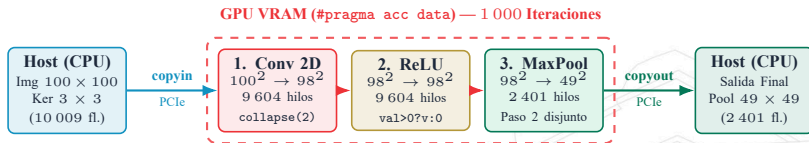
Juan Felipe Rodríguez | MCIC

Maestría en Ciencias de la
Información y las Comunicaciones

Computación Paralela y Distribuida
2026

Flujograma General de la Tubería y Memoria

Ciclo de vida Host-Device y persistencia en VRAM con OpenACC



1. Transferencias PCIe:

copyin y copyout operan una sola vez al inicio y al final de la ráfaga completa.

2. Encadenamiento

VRAM: Buffers conv y relu asignados con `create` con **cero** tráfico PCIe intermedio.

3. Coalescencia 1D:

Arreglos continuos 1D ($i * W + j$) para accesos óptimos en memoria global GPU.

Etapa 1: Convolución 2D (Flujograma y Paralelismo)

Filtrado espacial 3×3 colapsado en 9 604 hilos paralelos

Inicio de Kernel GPU: Hilo $(i, j) \in [0..97] \times [0..97]$
Malla de 9 604 hilos vía `#pragma acc parallel loop collapse(2)`

Inicializar acumulador local en
registro: `float sum = 0.0f`

Doble Bucle Espacial: $ki \in [0..2], kj \in [0..2]$
Leer píxel: `img[(i + ki) * 100 + (j + kj)]` × peso: `kernel[ki*3+kj]`
Acumular: `sum += píxel * peso`

Escribir en VRAM: `conv[i * 98 + j] = sum`

Fin de Hilo: 9 604 valores procesados en paralelo

Operación Aritmética:

- ▶ **Carga local:** 9 productos y 9 acumulaciones (18 FLOPs/píxel).
- ▶ **Total etapa:** $9\,604 \times 18 \approx 172.8$ kFLOPs/iteración.
- ▶ **Colapso 2D:** Fusión bidimensional que evita subutilizar la GPU ($98 \rightarrow 9\,604$ hilos).

```
#pragma acc parallel loop collapse(2)
for (int i = 0; i < 98; ++i) {
    for (int j = 0; j < 98; ++j) {
        float sum = 0.0f;
        for (int ki = 0; ki < 3; ++ki)
            for (int kj = 0; kj < 3; ++kj)
                sum += img[...] * kernel[...];
        conv[i * 98 + j] = sum;
    }
}
```

Etapa 2: Activación ReLU (Flujograma y Evaluación)

Truncamiento no lineal sin dependencias espaciales

Inicio de Hilo GPU: $(i, j) \in [0..97] \times [0..97]$
Malla de 9 604 hilos vía *collapse(2)*

Lectura desde VRAM: float
 $val = conv[i * 98 + j]$

¿ $val > 0.0f$?

Sí:
 $relu[...] = val$

No:
 $relu[...] = 0.0f$

Fin de Hilo: Salida guardada en memoria VRAM

Fundamentos de Activación:

- ▶ **Sin dependencias:** Operación puramente local e independiente elemento a elemento.
- ▶ **Preservación dimensional:** Entrada y salida idénticas de 98×98 (9 604 elementos).
- ▶ **Cero transferencias:** Consume conv en VRAM y produce relu en VRAM.

```
#pragma acc parallel loop collapse(2)
for (int i = 0; i < 98; ++i) {
    for (int j = 0; j < 98; ++j) {
        float val = conv[i * 98 + j];
        relu[i * 98 + j] = (val > 0.0f) ? val : 0.0f;
    }
}
```

Etapa 3: MaxPooling 2D (Flujograma y Reducción)

Submuestreo disjunto con paso 2 sin parámetros entrenables

Inicio de Hilo GPU: $(i, j) \in [0..48] \times [0..48]$
Malla de 2 401 hilos vía *collapse(2)*

Mapeo Stride 2 en buffer ReLU ($W = 98$):

$r_{00} = (2i) * 98 + 2j$, $r_{01} = (2i) * 98 + (2j+1)$
 $r_{10} = (2i+1) * 98 + 2j$, $r_{11} = (2i+1) * 98 + (2j+1)$

Lectura del vecindario 2×2 en buffer ReLU

Supremo local: $\max(\max(r_{00}, r_{01}), \max(r_{10}, r_{11}))$

Salida Compacta: $\text{pool}[i*49 + j] = \max(2\,401 \text{ puntos})$

Corrección Teórica (Ventana 2x2):

- La “ventana 2x2” **no es una matriz** ni posee pesos numéricos entrenables.
- Es únicamente un **rango de lectura espacial**: extrae el supremo de las 4 celdas continuas.

```
#pragma acc parallel loop collapse(2)
for (int i = 0; i < 49; ++i) {
    for (int j = 0; j < 49; ++j) {
        int r0 = (2*i)*98 + 2*j, r1 = (2*i+1)*98 + 2*j;
        float m0 = std::max(relu[r0], relu[r0 + 1]);
        float m1 = std::max(relu[r1], relu[r1 + 1]);
        pool[i*49 + j] = std::max(m0, m1);
    }
}
```

Estrategia de Memoria en GPU con OpenACC

Eliminación de transferencias intermedias en el bus PCIe

```
#pragma acc data \
  copyin(img[0:10000], kernel[0:9]) \
  create(conv[0:9604], relu[0:9604]) \
  copyout(pool[0:2401])
{
  for (int it = 0; it < 1000; ++it) {
    // 1. Convolucion 2D (GPU)
    // 2. Activacion ReLU (GPU)
    // 3. MaxPooling 2D (GPU)
  }
}
```

Análisis de Cláusulas de Datos:

- ▶ **copyin:** Transfiere la imagen y el kernel a la GPU **una sola vez** al inicio.
- ▶ **create:** Asigna buffers temporales en VRAM. **Cero transferencias PCIe** durante las 1,000 repeticiones.
- ▶ **copyout:** Retorna únicamente la salida compacta (49×49) al Host al culminar la ráfaga.

Resultados Experimentales y Ley de Amdahl

Aceleración neta de cómputo ($15.45\times$) y total de sistema ($13.08\times$)

Rendimiento (1 000 Iteraciones):

Métrica de Evaluación	Valor	Unidad
CPU Secuencial (T_{CPU})	32.46	ms
GPU Cómputo Neto (T_{Kernel})	2.10	ms
GPU Total con E/S (T_{Total})	2.48	ms
Overhead de bus PCIe	0.38	ms (15.3%)
Speedup Cómputo ($S_{\text{cómputo}}$)	15.45x	Aceleración
Speedup Total (S_{total})	13.08x	Aceleración

Consistencia: Verificación bit a bit exacta ($\Delta_{\text{max}} = 0.000$).

Análisis con la Ley de Amdahl:

- ▶ **Granularidad de 100×100 :** Carga moderada donde el costo de lanzamiento de kernels es perceptible.
- ▶ **Amortización del Bucle:** Con $N = 1$, el bus PCIe dominaría arrojando $S < 1\times$. Con $N = 1\,000$, el costo se amortiza a $0.38\ \mu\text{s}$ /iteración.
- ▶ **Escalabilidad:** Matrices mayores (1024×1024) saturan completamente los multiprocesadores CUDA.



Conclusiones y Trabajo Futuro

Lecciones aprendidas en HPC y próximos pasos

Conclusiones Principales:

- ▶ **Aceleración Masiva:** OpenACC permitió superar en más de $13\times$ a la CPU con directivas directas y portables.
- ▶ **Gestión de Datos:** El uso de `acc data` con `create` es el factor decisivo para evitar transferencias en el bus PCIe.
- ▶ **Colapso 2D:** La cláusula `collapse(2)` es indispensable para crear mallas de alta ocupación (9 604 hilos).

Trabajo Futuro:

- ▶ **Fusión de Kernels:** Integrar Convolución y ReLU en un solo kernel para procesar en registros y ahorrar accesos a VRAM.
- ▶ **Soporte Multicanal:** Extender la tubería a tensores 4D (batch, canales, alto, ancho) de modelos profundos (ResNet, VGG).
- ▶ **Memoria Compartida:** Aplicar directivas `cache` para reusar píxeles adyacentes en la convolución.



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**
Acreditación Institucional en Alta Calidad

www.udistrital.edu.co

VIGILADO MINEDUCACIÓN.
Diseño: Unidad de Publicaciones
Universidad Distrital Francisco José de Caldas

¡Muchas Gracias!

Espacio para Preguntas y Discusión

Juan Felipe Rodríguez

Maestría en Ciencias de la Información
y las Comunicaciones

Computación Paralela y Distribuida